Quiz Module — Reference, Stories & Issues

Covers `src/modules/quiz/**`, its question selector `src/modules/question/providers/question-generator.service.ts`, and the Angular screens that call it. Last reviewed: **2026-10-14** (full review after the Quick Practice outage, see §8.1).

Contents

1. Overview
2. Data model
3. API endpoints
4. User stories & scenarios
5. How it works
6. Error codes
7. Constants
8. Issues register
9. Enhancement backlog
10. Frontend touchpoints

---

1. Overview

There are two kinds of quiz. They are stored in **separate tables** with the same column layout:

Type	Table	Who picks the questions	Used for
UserQuiz (practice)	`user_quiz`	The server (`QuestionGeneratorService`), from the learner's history	Quick Practice, topic/subject/track practice, guided "keep going" quizzes, the onboarding assessment
Standard (catalog)	`quiz`	An admin/moderator, via an explicit `questionIds` list	Curated, published quizzes in the Browse Quizzes catalog

Rules that apply to both:

- **One attempt per person per quiz.** A quiz can be shared and taken by many people, but each person submits once. A second submit returns `409 QUIZ_ALREADY_TAKEN`.
- **The server grades.** Correctness, totals and score are always recomputed from `QuestionOption`; client-sent values are ignored.
- **Results are public by code.** `GET /result/:resultCode` is shareable and exposes display name/username/avatar only.

The two tables have **independent id sequences**: `quiz.id = 7` and `user_quiz.id = 7` are different quizzes. So any call that names a quiz by id must also send `quizType`. Slugs, by contrast, are kept unique across both tables.

2. Data model

```
quiz (Standard)                 user_quiz (UserQuiz)
 ├─< quiz_question >─ question   ├─ questionIds  (JSON array, inline)
 ├─< quiz_subject  >─ subject    ├─< quiz_subject  (userQuizId)
 ├─< quiz_topic    >─ topic      ├─< quiz_topic    (userQuizId)
 ├── quiz_settings (0..1)        ├── quiz_settings (0..1, onboarding only)
 └─< quiz_result ─< question_attempt   └─< quiz_result ─< question_attempt
```

- **Link tables** (`quiz_question` , `quiz_subject` , `quiz_topic` , `quiz_settings` , `quiz_result` , `question_attempt`) carry **exactly one of** `quizId` / `userQuizId` , plus a denormalised `quizType` . Both FKs are `ON DELETE SET NULL` , so results outlive a purged practice quiz.
- **UserQuiz stores its question list inline** (`user_quiz.questionIds` , `simple-json`). It does not use `quiz_question` .
- `quiz_result` has unique indexes `(quizId, userId)` and `(userQuizId, userId)` . These enforce the one-attempt rule under concurrent submits.
- `quiz_result.resultCode` is a random 6-digit string with a unique index (see OI-3).
- `QuizResult` and `QuestionAttempt` declare **eager** relations (`quiz` , `userQuiz` , `user`). This matters for queries; see §5.8.
- `quiz.createdBy` / `user_quiz.createdBy` are `select: false` . Select them explicitly when needed.
- `Question` ↔ `Topic` is many-to-many (`question_topic`), so joining topics fans out rows. The result page and the generator both guard against this.
- `DifficultyLevelEnum` : `Easy = 1` , `Intermediate = 2` , `Advanced = 3` .

3. API endpoints

Base path `apis/quiz` . A global `JwtAuthGuard` applies unless a route is `@Public()` .

Method & path	Auth	Purpose	Notable responses
`GET /fetch/:slug`	Public, optional JWT	Quiz + questions. UserQuiz questions ordered Easy→Advanced (random within a level). Signed-in viewers who already took it get `myResultCode` .	404 not found
`POST /create`	JWT	Create a UserQuiz (anyone) or Standard quiz (Admin/Moderator only).	201 `{ message, data: quiz }` ; 403 `SUBJECT_NOT_ENROLLED` / `DAILY_QUOTA_EXCEEDED` ; 404 `SCOPE_EMPTY` / `QUESTIONS_EXHAUSTED` ; 400 reserved tag
`POST /submit`	JWT	Submit attempts. `userId` and IP come from the server.	409 `QUIZ_ALREADY_TAKEN` ; 400 no matching answers / missing `quizType`
`GET /result/:resultCode`	Public, optional JWT	Full breakdown by subject, topic and question; `isOwner` for the viewer.	404
`PUT /update/:quizId`	JWT	Edit a Standard quiz. **Original author only.**	403 / 404 / 400
`GET /quizzes`	Public, optional JWT	Published Standard catalog. Filters: `subjectId` , `topicId` , `jobRoleId` , `mode` (`0` = no filter). Signed-in callers get `isQuizTaken` , `resultCodes` , `latestResultCode` .	400 invalid filter

Method & path	Auth	Purpose	Notable responses
`GET /me`	JWT	"My Quizzes": practice quizzes the caller generated.	
`GET /quota/subject/:subjectId`	JWT	Today's practice-question allowance for one subject (display only).	
`GET /:userId`	JWT	Standard quizzes the user authored, drafts included. Non-admins can only ask for themselves; admins see all authors.	403

4. User stories & scenarios

Each story gives the user goal, the flow through the system, and the edge cases that matter.

Story 1 — Quick Practice from the Skill Arena

> *As a learner on my dashboard, I want one click to give me a short practice quiz on what I should study next, so I keep momentum without choosing a topic myself.*

Flow: `skill-arena.component` → `POST /create` (`quizType: UserQuiz` , a seed topic, usually `continueThroughSubject: true`) → `GET /fetch/:slug` → questions render inline.

Scenario	What happens
Normal case	Up to 10 new questions (never answered correctly), easiest first.
Learner has answered every question in the subject correctly	`404 QUESTIONS_EXHAUSTED` . The arena shows a completion milestone, not an error.
Subject has published topics but no Active Trivia questions	`404 SCOPE_EMPTY` ("No content available yet").
Not enrolled in the subject	`403 SUBJECT_NOT_ENROLLED` → enroll prompt.
Free/Basic tier, today's allowance used	`403 DAILY_QUOTA_EXCEEDED` → upgrade upsell.
Fewer new questions exist than requested	Quiz is created with what exists; the message says "*N of M requested*".
Fetch fails	Arena shows "*Could not load your practice questions*". Before 2026-10-14 this happened on **every** signed-in fetch (§8.1, F-1).

Story 2 — Guided "keep going" practice through a subject

> *As a learner, when a topic runs out of new questions I want the quiz to keep going into the next topics, so I'm not stopped by a dead end.*

Flow: caller sends a single `topicIds` value plus `continueThroughSubject: true` . The server expands it to that topic **plus every later published topic in the same subject** (`Topic.order` , then id). It never crosses into another subject. The generator then drains topics in that order (§5.2).

Scenario	What happens
Seed topic has 3 new questions, quiz wants 10	3 from the seed topic, 7 from the next topics.
Seed topic was deleted meanwhile	Falls back to just the seed id.
More than one topic, or flag false	No expansion. The quiz stays on exactly the given scope (used by "practice this topic").

Story 3 — Practice a specific topic, subject or track

> *As a learner browsing a subject's curriculum, I want to practice one topic, a whole subject, or a track.*

Flow: `subject-curriculum-page` → `quiz-create` dialog → `POST /create` with `topicIds`, `subjectIds` and/or `subjectTrackIds` (comma-separated strings). Scopes **combine** (union of topics); they don't override each other. The title and short description are auto-generated when blank. The first scope present wins, in the order Subject → SubjectTrack → Topic. A slug like `html-forms-quiz-by-vishal-kumar-md41k2a0` makes every practice quiz URL unique.

Story 4 — Free-tier daily question allowance

> *As a free/Basic learner, I can practice a limited number of questions per subject each day. As a Pro learner, I practice without limits.*

Scenario	What happens
Pro tier or above on every subject in scope	No cap.
Basic tier, 40/50 used on Subject A	Allowed. The new quiz's questions count toward A.
Basic tier, 50/50 used on Subject A	`403 DAILY_QUOTA_EXCEEDED`.
Quiz spans Subjects A (exhausted) and B (fresh)	**Blocked**: any exhausted subject blocks the quiz. See OI-6.
Learner at 45/50 starts a 25-question quiz	**Allowed.** The check doesn't look ahead, so the day ends at 70/50 (OI-5).
Learner generates quizzes but never takes them	Each still counts. The cap counts questions in quizzes **generated** today, not answered (OI-5).
Subject dashboard shows "X of Y left today"	`GET /quota/subject/:id`; the same counting as the check.

Story 5 — Taking and submitting a quiz

> *As a learner, I answer the questions and get my score, XP and any badges straight away.*

Flow: `take-quiz.component` → `GET /fetch/:slug` → answers → `POST /submit` (`quizId` + `quizType` + attempts). On tab close, the submit is sent with `fetch(..., { keepalive: true })` so an unload doesn't lose it.

Server-side, in order:

1. Load the quiz from the table named by `quizType`. A bare id is never guessed.
2. Reject if this person already has a result (`409`).
3. Drop attempts for questions not in the quiz, and duplicates (first wins). If none are left → `400`.
4. Re-grade every option-based question against `QuestionOption.correct`.
5. Compute total/correct/wrong/unanswered and score with `generateScore()`. Generate the feedback sentence.
6. **One transaction:** `quiz_result` + all `question_attempt` rows.
7. **After commit, never failing the submission:** notification, "Quiz Completed" activity, achievements (XP, streak, badges, certificates).

Scenario	What happens
Double-click / two tabs submit at once	The unique index rejects the second → `409`, the same as the pre-check.
Client claims a wrong answer is correct	Overridden by the server re-grade.
Question has options but none marked correct (bad data)	Graded incorrect, logged as a warning.
Free-text question (no options) on a Standard quiz	Client's `isCorrect` is kept; the server has nothing to check against.
Badge engine throws	Result is still saved; the error is logged.

Story 6 — Opening a shared quiz link

> *As a visitor who received a quiz link, I can open it, and I'm asked to register before my answers are submitted.*

`/quiz/take/:qcode` is reachable without a session (the fetch is `@Public`). On submit the page runs `quickRegister()`, then submits. A signed-in viewer who already took the quiz is sent straight to their result via `myResultCode`, instead of playing a quiz the server would reject.

Story 7 — Sharing a result

> *As a learner, I share my result link; anyone can open it and see my score breakdown.*

`GET /result/:code` returns score, accuracy, per-subject and per-topic stats, and every question with its options, the learner's choice and the correct answer. It exposes name, username and avatar only, never user id, email, IP or device. `isOwner` lets the page show owner-only actions.

How the stats are counted:

- **Subjects** count each question once.
- **Topics** count a multi-topic question once **per topic**.
- **Accuracy** = correct / asked (skips count as asked), matching the dashboards.

Story 8 — My Quizzes (practice history)

> *As a learner, I see every practice quiz I generated and can jump to my result.*

`GET /me`: lists UserQuizzes the caller **created**, with question count, total attempts (including other people the quiz was shared with) and the caller's **own** latest result. `isQuizTaken` reflects the caller only.

Scenario	What happens
A friend took a quiz I generated but I haven't	Shows as **not taken** by me; `totalAttempts` = 1. (Before 2026-10-14 this showed as taken, linking to the friend's result — F-5.)
I took a quiz a friend generated	Not listed. The list is creator-scoped (OI-15).

Story 9 — Onboarding initial assessment

> *As a newly registered user, I get a one-time skill check across my job role's subjects.*

`createInitialAssessmentQuiz(userId, firstName, jobRoleId)`, called from the users controller:

- **Idempotent:** if the user already has a quiz tagged `initial_assessment`, it is returned unchanged.
- **Scope:** every subject in `job_role_subject` for the role, `INITIAL_ASSESSMENT_LENGTH` (20) questions. It can be fewer if the content is thin.
- **Not gated:** it skips the enrollment/daily-cap check through an internal `skipAccessCheck` option. The tag `initial_assessment` is **reserved**: the API rejects it from clients (F-10).
- Gets a `quiz_settings` row with `maxAttempts = 1`. The one-attempt rule already guarantees that (OI-7).

Story 10 — Admin authors and edits a catalog quiz

> *As an admin/moderator, I build a quiz from hand-picked questions, publish it, and edit it later.*

Flow: `quiz-builder.component` → `POST /create` (`quizType: Standard`, `questionIds`, title/descriptions required, optional `settings`, `isPublished`) → edits via `PUT /update/:id`.

Scenario	What happens
A learner sends `quizType: Standard`	`403`.
Edit with `subjectIds: ""`	Subject links are cleared. Omitting the field leaves them unchanged.
Edit with a new `questionIds` list	The question list is replaced wholesale.
Rename to a title whose slug exists in `user_quiz`	A unique suffix is generated. (Before 2026-10-14 only `quiz` was checked — F-7.)
A different admin tries to edit	`403`. Only the original author can (OI-9).
`numQuestions` smaller than `questionIds.length`	Ignored; every listed question is kept (OI-8).

Story 11 — Browsing the published catalog

> *As a learner (or visitor), I browse published quizzes filtered by job role, subject, topic or mode, and see which ones I've already taken.*

`GET /quizzes?jobRoleId=&subjectId=&topicId=&mode=`. Filters are `EXISTS` subqueries, so they never duplicate rows or hide a quiz's other subjects. Signed-in callers get `isQuizTaken` and their result codes. Before 2026-10-14, any `topicId` filter returned 500 (F-2).

Story 12 — Authored-quiz list

> *As a quiz author, I see my Standard quizzes, drafts included, with question and attempt counts.*

`GET /:userId`. Non-admins can only request their own id (F-6). Admins get every author's quizzes regardless of the id.

5. How it works

5.1 Creation pipeline (`QuizService.createQuiz`)

1. Validate: at least one scope; Standard needs `questionIds`.
2. Parse comma-separated id lists (`parseIdList`). Apply `continueThroughSubject` expansion.
3. **Gate** UserQuizzes, unless `skipAccessCheck` is set (§5.3).
4. Resolve the length: Standard = `min(numQuestions, questionIds.length)` (only informational, OI-8); UserQuiz = `min(numQuestions ?? 10, 25)`.
5. Select questions: Standard via `QuestionService.getQuestionsFromQIds`; UserQuiz via `QuestionGeneratorService.generateUserQuiz` (§5.2). Zero questions → 404.
6. Build the title, short description and a slug that is unique across both tables.
7. **One transaction:** the quiz row (UserQuiz includes `questionIds`), `quiz_question` (Standard only), `quiz_subject`, `quiz_topic`, `quiz_settings` (Standard, if given). Failures are logged and returned as a generic 500.

Response `data`: the saved quiz plus `quizType`. UserQuizzes also get `requestedCount` / `actualCount`.

5.2 Practice question selection (`QuestionGeneratorService.generateUserQuiz`)

- **Scope → ordered topic list.** Subjects and tracks expand to their **published** topics. Explicit `topicIds` pass through. Everything is sorted by `Topic.order`, then id.
- **Difficulty gate per topic.** A tier is "cleared" once the learner has answered at least `TIER_CLEAR_COVERAGE_PERCENT` (70%) of **that topic's own** questions at that tier correctly. A tier with no questions counts as cleared. The topic's cap is the next uncleared tier: Easy → Intermediate → Advanced.
- **Sequential drain.** The selector walks topics in order and takes as many questions as are still needed from each one: Active Trivia, `level ≤ cap`, **never answered correctly by this user**, not already picked. Order is `level,`

`orderId, id`, so it is **deterministic** (no randomness).

- **No padding.** If the scope runs dry, the quiz is shorter and the message says so. Zero results → `SCOPE_EMPTY` (no content at all) or `QUESTIONS_EXHAUSTED` (all answered correctly).
- Consequences of this design:
  - A wrongly answered question is eligible again immediately.
  - Generating the same scope twice without taking it gives the same questions.
  - The fetch step then shuffles within each difficulty level.

5.3 Enrollment & daily cap (`enforceSubjectAccessForUserQuiz`)

1. Resolve the request to the set of target subjects (topics → subject; tracks → their topics' subjects).
2. Look up the learner's **lowest** effective tier across those subjects. If they're not enrolled in any one of them → `403 SUBJECT_NOT_ENROLLED`.
3. Pro and above → allowed.
4. Otherwise read `dailyQuestionCap` for that tier. For each subject, sum `JSON_LENGTH(user_quiz.questionIds)` over the learner's practice quizzes **created since server-local midnight**. If any target subject is at or over the cap → `403 DAILY_QUOTA_EXCEEDED`.

`getSubjectQuestionQuota` uses the same counting for display.

5.4 Fetching (`fetchQuizBySlug`)

- Looks in `quiz` first, then `user_quiz`.
- Question ids come from `quiz_question` (Standard) or `user_quiz.questionIds`.
- Loads full questions with options and topics.
- Orders UserQuiz questions by level, random within a level.
- For a signed-in viewer, attaches `myResultCode` if they already submitted.

⚠ The response includes each option's `correct` flag and the `answer` text (OI-2).

5.5 Submission (`submitQuiz`)

See Story 5 for the full order of operations.

- Score uses the shared `generateScore(total, correct, wrong)`.
- Pass marks come from `quiz_settings.passMarks`, defaulting to 60.
- The feedback sentence comes from `generateQuizResultFeedback`.

5.6 Results (`QuizResultService.getQuizResultByCode`)

- One base query: result + user + `COALESCE`d quiz/userQuiz title, description and slug.
- Attempts are scoped by `resultId`. Legacy rows with no `resultId` fall back to `userId` + quiz.
- Options for all questions are loaded in **one** query.
- Aggregates follow the rules in Story 7.
- `selectedOptionId` is `null` for skipped questions.

5.7 Lists

- **`getPublishedQuizzes`:** entity query plus two batched count queries (attempts, questions).
- **`getMyPracticeQuizzes`:** one grouped query with a correlated subquery for the owner's latest result.
- **`getUserQuizzes`:** grouped `COUNT(DISTINCT)` over results × questions (OI-13).

5.8 TypeORM gotcha that caused the 2026-10-14 outage

On an entity with **eager relations** (`QuizResult` , `QuestionAttempt` , `JobRoleSubject` , `AssessmentSession` , `UserOtp`), `find` / `findOne` with a `select` builds a DISTINCT subquery keyed on the primary key. If `id` isn't selected, MySQL fails with `Unknown column 'distinctAlias.<Entity>_id'` . When selecting columns on these entities:

- always include `id` , and
- pass `loadEagerRelations: false` if the relations aren't needed.

A sweep of the whole API on 2026-10-14 found no other instance.

6. Error codes

HTTP	`code`	Raised by	Meaning / UI treatment
403	`SUBJECT_NOT_ENROLLED`	create	Not enrolled in a target subject → "Enroll Now"
403	`DAILY_QUOTA_EXCEEDED`	create	Free-tier daily questions used → upgrade upsell
404	`SCOPE_EMPTY`	create	Scope has no Active Trivia content
404	`QUESTIONS_EXHAUSTED`	create	Learner has answered everything correctly → milestone
409	`QUIZ_ALREADY_TAKEN`	submit	One attempt per person per quiz
400	—	create	Reserved tag, missing scope, Standard without `questionIds`
400	—	submit	Missing `quizType` ; no attempts match the quiz
500	—	any	Server fault. The message is generic and the full error is in the API console (`GlobalExceptionFilter`).

7. Constants

`src/common/constants/quiz-generation.constants.ts`

Constant	Value	Meaning
`DEFAULT_QUIZ_LENGTH`	10	Practice length when `numQuestions` is omitted
`MAX_QUIZ_LENGTH`	25	Hard ceiling for practice quizzes, enforced in the service and the generator
`INITIAL_ASSESSMENT_LENGTH`	20	Onboarding assessment length (must be ≤ `MAX_QUIZ_LENGTH`)
`TIER_CLEAR_COVERAGE_PERCENT`	70	% of a topic's own tier pool needed to unlock the next tier
`INITIAL_ASSESSMENT_TAG`	`'initial_assessment'`	Reserved `user_quiz.tag` (in `quiz.service.ts`)

8. Issues register

8.1 Fixed in the 2026-10-14 review

#	Severity	Issue	Fix
F-1	**Critical**	`GET /fetch/:slug` returned 500 for **every signed-in user**, so Quick Practice and every quiz page failed. The prior-result lookup used `findOne({ select: ['resultCode'] })` on eager-relation `QuizResult` (§5.8). Introduced 2026-10-13 ("Quiz and Invitation Enhancements").	Select `id` too; `loadEagerRelations: false` .
F-2	High	`GET /quizzes?topicId=…` always returned 500: the filter referenced the nonexistent column `job_role_subject.topicId` .	Filters rewritten as `EXISTS` subqueries.

#	Severity	Issue	Fix
F-3	**High (security)**	The enrollment/daily-cap gate was skipped for any client-sent `tag: 'initial_assessment'`. Learners could practice unenrolled premium subjects with no cap. Planting the tag also blocked the user's real onboarding assessment (idempotency lookup). No abuse found in the dev DB.	Gate skip is now an internal `skipAccessCheck` option; the API rejects the reserved tag.
F-4	Medium	`createQuiz` returned the transaction promise without `await` inside `try`. DB failures bypassed the catch, so the friendly error and log never fired.	`return await`.
F-5	Medium	"My Quizzes" marked a shared practice quiz as taken (and linked the result) when **anyone** had taken it.	`latestResultCode` and `isQuizTaken` scoped to the quiz owner.
F-6	Medium (privacy)	`GET /:userId` let any signed-in user list another author's quizzes, **drafts included**.	Non-admins restricted to their own id (403 otherwise).
F-7	Medium	Editing a Standard quiz's title only checked slug uniqueness in `quiz`. A collision with a `user_quiz` slug would make one of them unreachable.	Shared `slugExists()` checks both tables.
F-8	Low (latent)	Result page per-subject stats counted a multi-topic question once per topic. Not present in current data.	Subject stats use one row per question.
F-9	Low	Result page returned `selectedOptionId: 0` for skipped questions (`Number(null)`), and ran one options query per question.	`null` for skipped; one batched options query.
F-10	Low	Dead code: unused `QuestionAttemptService` (with an unbounded analytics query), `findQuizIdsTakenByUser`, the duplicate `topic-list.dto.ts`, unused `MailService` / `QuizTopic` repo injections. Frontend `getAllQuiz()` / `deleteQuiz()` called routes that don't exist.	Removed.
F-11	Low	Type/comment drift: `createQuiz` was declared `Promise<Quiz>` but returned `{message, quiz}`. Comments claimed the cap counted questions "answered" and that title precedence was "most specific wins" (it's first-present-wins). An unreachable third quiz-type branch. Long history-style comments throughout.	Types corrected, comments rewritten to describe current behaviour, dead branch removed.
F-12	Low	`PublishedQuizFilterDto` had no validation (`?subjectId=abc` reached SQL as `NaN`).	Validated; invalid values → 400.

8.2 Open issues

#	Severity	Area	Issue	Impact	Suggested fix
OI-1	**High**	Security (auth, app-wide)	The JWT secret is hardcoded as `'secret@1234#'` in `auth.module.ts` (signing) and `jwt.strategy.ts` (verification). `JWT_SECRET` in `.env` is unused.	Anyone with repo access can mint a valid token for any user, including admins.	Read the secret from config in both places; rotate it (this invalidates current sessions).
OI-2	**High**	Integrity	`GET /fetch/:slug` sends every option's `correct` flag and the `answer` text **before** submission.	Answers are readable in devtools, and XP, badges and **certificates** derive from these scores.	Strip `correct` / `answer` from fetch. Add a per-question check endpoint (or reveal on submit) for instant feedback. Needs a take-quiz change.

#	Severity	Area	Issue	Impact	Suggested fix
OI-3	Medium	Reliability / privacy	`resultCode` is a random 6-digit number with a unique index and **no retry**. A collision fails the submission with a generic 500. P(fail) ≈ results ÷ 900,000 per submit: ~1% at 9k results, ~10% at 90k. Codes are also enumerable in a 900k space.	Lost submissions as data grows; public result pages are guessable.	Longer non-sequential code (e.g. 10-char base36), plus retry on duplicate.
OI-4	Medium	Validation (app-wide)	Global `ValidationPipe` has neither `transform` nor `whitelist`. DTO default values never apply (e.g. `QuizSettingsDto` defaults, `CreateQuizDto.category`). Unknown fields pass through to entities: `settings.id` in `PUT /update` would retarget another quiz's settings row. Staff-only routes.	Misleading defaults; mass-assignment risk.	Enable `whitelist: true` (test for clients relying on extra fields), or pick fields explicitly when building entities.
OI-5	Medium	Product (daily cap)	The cap counts questions in quizzes **generated** today, not answered. 80 of 152 practice quizzes in dev were never taken but still consumed allowance. No look-ahead (45/50 can start a 25-question quiz). The day boundary is **server-local midnight**, not the learner's timezone.	Unfair throttling; cap slightly leaky; reset time differs from the learner's day.	Count from `question_attempt`; check `used + requested ≤ cap`; use a defined timezone (UTC or user profile).
OI-6	Medium	Product (daily cap)	A quiz spanning several subjects is blocked if **any** of them is exhausted, even when the others have allowance. Options: keep; exclude the exhausted subject's topics from the quiz; charge only the primary subject.	Cross-subject quizzes are harder to take than single-subject ones.	Product decision. The per-subject summing is in place; only the blocking condition changes.
OI-7	Low	Dead setting	`QuizSettings.maxAttempts` is superseded by the one-attempt rule and never read. The builder may still expose it; the onboarding assessment writes it.	Misleading admin setting.	Remove from DTO/UI, or implement retakes deliberately.
OI-8	Low	Dead parameter	`numQuestions` is a no-op for Standard quizzes: every listed question is kept.	Confusing API.	Drop it for Standard, or actually trim.
OI-9	Low	Admin workflow	Only a quiz's original author can edit it (other admins get 403). No delete/unpublish-by-other endpoint. No purge job for abandoned practice quizzes, so `user_quiz` grows unbounded.	Admin friction; table growth.	Allow Admin role to edit; add delete/archive; schedule a purge of untaken practice quizzes older than N days.
OI-10	Low	Consistency	The onboarding assessment's settings row is saved outside the `createQuiz` transaction.	A crash in between leaves an assessment without settings. Harmless while OI-7 stands.	Fold into the create transaction, or drop the row with OI-7.
OI-11	Low	Typing / content	`generateUserQuiz` is declared to return `QuestionListResponseDto[]` but returns `{message, questions, requestedCount, actualCount}`. Explicit `topicIds` aren't checked for `isPublished` (subject/track expansion is).	Type drift; unpublished topics can be practiced by id.	Fix the return type; filter explicit topics to published.

#	Severity	Area	Issue	Impact	Suggested fix
OI-12	Low	Learning design	Selection is deterministic with no cooldown. A just-missed question is served again immediately, and re-generating the same scope gives identical quizzes.	Repetitive practice on small topics.	Optional one-quiz cooldown for wrong answers, or light randomisation within a level.
OI-13	Low	Performance	`getUserQuizzes` uses `COUNT(DISTINCT)` over a results × questions join.	Slows as attempts grow.	Two grouped subqueries, as `getPublishedQuizzes` does.
OI-14	Low	Data	The "Quiz Completed" notification stores a bare `quizId` without the quiz type (ambiguous across tables). Unused by the UI today.	Wrong link if notifications become clickable.	Store `resultId` / `resultCode` instead.
OI-15	Low	UX	"My Quizzes" lists only quizzes the learner **created**. Shared quizzes they took elsewhere don't appear.	Incomplete history.	Union in quizzes the learner has results for.

9. Enhancement backlog

Ordered roughly by value. Items marked **(fixes OI-n)** close an open issue.

1. **Server-side answer reveal** (fixes OI-2). Keep answers server-side until submit, or check per question. This is the prerequisite for trusting quiz-earned certificates.
2. **Config-driven JWT secret + rotation** (fixes OI-1).
3. **Robust result codes** (fixes OI-3): longer codes plus retry on duplicate key.
4. **Fairer daily cap** (fixes OI-5, OI-6): count answered questions, look ahead, use a defined timezone, and settle the multi-subject rule.
5. **Practice-quiz lifecycle** (OI-9): purge untaken UserQuizzes after N days. Results survive via `ON DELETE SET NULL`.
6. **Spaced repetition lite** (OI-12): cool down wrong answers for one quiz, then resurface them deliberately ("review mistakes" mode).
7. **Admin catalog management** (OI-9, OI-7): any admin can edit; delete/archive; remove or implement `maxAttempts`.
8. **Global `ValidationPipe` hardening** (OI-4): `whitelist: true` across the API, with a compatibility check.
9. **Complete history** (OI-15): "Quizzes I've taken", not only "quizzes I made".
10. **Regression tests** for the cases that broke in production-like use:
    - signed-in fetch
    - topic filter
    - reserved tag
    - double-submit 409
    - multi-topic result stats
    - the eager-relation `select` pattern (§5.8)

10. Frontend touchpoints

Screen / component (codemerit)	Endpoints	Notes
`shared/components/skill-arena` (Quick Practice)	create → fetch	Handles `SUBJECT_NOT_ENROLLED`, `DAILY_QUOTA_EXCEEDED`, `QUESTIONS_EXHAUSTED`

Screen / component (codemerit)	Endpoints	Notes
`shared/components/quiz-create` (practice dialog)	create	Opened from the subject curriculum page and the next-quiz widget
`shared/components/next-quiz-widget`	create (`continueThroughSubject`)	Guided "keep going"
`subscriber/subject-curriculum-page`	create via dialog, quota	
`quiz/take-quiz` (`/quiz/take/:qcode`)	fetch, submit (keepalive)	`myResultCode` redirect, `quickRegister()` for visitors, 409 handling
`quiz/view-result` (`/quiz/result/:qcode`)	result	Public share page
`pages/my-quizzes`, `lms/quizzes/myQuizzes`	me, `/:userId`	
`pages/browse-quizzes`	quizzes	Catalog filters
`quiz/quiz-builder` (`/quiz/builder[/:slug]`)	create (Standard), update, fetch	Admin/Moderator
`subscriber/dashboard`	various	Entry points

All calls go through `src/app/quiz/quiz.service.ts`.